

Breast Cancer: K-Means and Random Forests

Alexis Alberto Zuniga Alonso

2026-10-07

Table of contents

1. Introduction	1
2. Data Wrangling	2
2.1 Loading the raw data	2
2.2 First look: structure and column names	3
2.3 Data types and value ranges	4
2.4 Missing values	6
2.5 Duplicated records	7
2.6 Coding the variables	8
2.7 Final check	9
3. Initial Statistical Description	10
3.1 Class distribution	10
3.2 Summary statistics of the predictors	10
3.3 Predictors by tumour class	11
3.4 Correlations between predictors	13
4. K-Means Algorithm	15
4.1 Fitting the model	15
4.2 Comparison with the original classes	16
4.3 Visualisation	17
5. Random Forest	19
5.1 Train / test split	20
5.2 Fitting the model	21
5.3 Variable importance	22
6. Performance Metrics	22
7. Discussion	24
7.1 Best and worst performance	24
7.2 Interpretation	24

1. Introduction

Consider the dataset `BreastCancer.csv` which contains 699 observations of clinical cases where some tumor related to breast cancer was detected, and it is sought to identify whether the tumor is benign or malignant. Additionally, the data of 10 variables was registered:

1. `Id` (char)
2. `Cl.thickness` (ordinal)
3. `Cell.size` (ordinal)
4. `Cell.shape` (ordinal)
5. `Marg.adhesion` (ordinal)
6. `Epith.c.size` (ordinal)
7. `Bare.nuclei` (factor)
8. `Bl.cromatin` (factor)
9. `Normal.nucleoli` (factor)
10. `Mitoses` (factor)

And the variable `Class` registers the type of tumor ('benign' or 'malignant').

2. Data Wrangling

2.1 Loading the raw data

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1    ✓ readr      2.2.0
✓ forcats    1.0.1    ✓ stringr    1.6.0
✓ ggplot2    4.0.3    ✓ tibble     3.3.1
✓ lubridate  1.9.5    ✓ tidyr      1.3.2
✓ purrr      1.2.2
— Conflicts — tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
  conflicts to become errors
```

```
library(janitor)
```

```
Attaching package: 'janitor'
```

```
The following objects are masked from 'package:stats':
```

```
  chisq.test, fisher.test
```

```
library(knitr)
```

```
raw <- read_csv("../data/raw/BreastCancer - BreastCancer.csv",
                show_col_types = FALSE)
```

New names:

- `` -> `...1`

```
dim(raw)
```

```
[1] 699 12
```

```
glimpse(raw)
```

```
Rows: 699
Columns: 12
$ ...1      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,...
$ Id        <dbl> 1000025, 1002945, 1015425, 1016277, 1017023, 1017122, ...
$ Cl.thickness <dbl> 5, 5, 3, 6, 4, 8, 1, 2, 2, 4, 1, 2, 5, 1, 8, 7, 4, 4, ...
$ Cell.size  <dbl> 1, 4, 1, 8, 1, 10, 1, 1, 1, 2, 1, 1, 3, 1, 7, 4, 1, 1,...
$ Cell.shape <dbl> 1, 4, 1, 8, 1, 10, 1, 2, 1, 1, 1, 1, 3, 1, 5, 6, 1, 1,...
$ Marg.adhesion <dbl> 1, 5, 1, 1, 3, 8, 1, 1, 1, 1, 1, 1, 3, 1, 10, 4, 1, 1,...
$ Epith.c.size <dbl> 2, 7, 2, 3, 2, 7, 2, 2, 2, 2, 1, 2, 2, 2, 7, 6, 2, 2, ...
$ Bare.nuclei <dbl> 1, 10, 2, 4, 1, 10, 10, 1, 1, 1, 1, 1, 3, 3, 9, 1, 1, ...
$ Bl.cromatin <dbl> 3, 3, 3, 3, 3, 9, 3, 3, 1, 2, 3, 2, 4, 3, 5, 4, 2, 3, ...
$ Normal.nucleoli <dbl> 1, 2, 1, 7, 1, 7, 1, 1, 1, 1, 1, 1, 4, 1, 5, 3, 1, 1, ...
$ Mitoses     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 5, 1, 1, 1, 1, 1, 4, 1, 1, 1, ...
$ Class       <chr> "benign", "benign", "benign", "benign", "benign", "mal...
```

2.2 First look: structure and column names

```
names(raw)
```

[1] "...1"	"Id"	"Cl.thickness"	"Cell.size"
[5] "Cell.shape"	"Marg.adhesion"	"Epith.c.size"	"Bare.nuclei"
[9] "Bl.cromatin"	"Normal.nucleoli"	"Mitoses"	"Class"

```
head(raw, 5)
```

```
# A tibble: 5 × 12
  ...1      Id Cl.thickness Cell.size Cell.shape Marg.adhesion Epith.c.size
  <dbl>   <dbl>         <dbl>    <dbl>    <dbl>         <dbl>         <dbl>
1     1 1000025           5         1         1           1           2
2     2 1002945           5         4         4           5           7
3     3 1015425           3         1         1           1           2
4     4 1016277           6         8         8           1           3
5     5 1017023           4         1         1           3           2
# i 5 more variables: Bare.nuclei <dbl>, Bl.cromatin <dbl>,
#   Normal.nucleoli <dbl>, Mitoses <dbl>, Class <chr>
```

Problems detected

- The first column has **no name** (...1) and is just a row counter exported from R. It carries no information.
- Column names mix capitals and dots (Cl.thickness, Epith.c.size), which makes them awkward to use in code.
- Id was read as a number, but it is an identifier, not something to do arithmetic with.

```
# Is the unnamed column really just 1..n?
all(raw[[1]] == seq_len(nrow(raw)))
```

```
[1] TRUE
```

It is a pure row counter, so it is safe to drop. Then the names are standardised (snake_case), and Id becomes a character.

```
df <- raw |>
  select(-1) |>
  clean_names() |>
  rename(bl_chromatin = bl_cromatin) |> # fix the typo in the original name
  mutate(id = as.character(id))

names(df)
```

```
[1] "id"          "cl_thickness" "cell_size"   "cell_shape"
[5] "marg_adhesion" "epith_c_size" "bare_nuclei" "bl_chromatin"
[9] "normal_nucleoli" "mitoses"      "class"
```

2.3 Data types and value ranges

```
sapply(df, class)
```

```

      id      cl_thickness      cell_size      cell_shape      marg_adhesion
"character" "numeric"      "numeric"      "numeric"      "numeric"
epith_c_size bare_nuclei    bl_chromatin normal_nucleoli      mitoses
"numeric"   "numeric"      "numeric"      "numeric"      "numeric"
class
"character"

```

```

# Frequency of every value of every predictor (long format)
df |>
  select(-id, -class) |>
  pivot_longer(everything(), names_to = "variable", values_to = "value") |>
  count(variable, value) |>
  pivot_wider(names_from = value, values_from = n, values_fill = 0) |>
  kable()

```

variable	1	2	3	4	5	6	7	8	9	10	NA
bare_nuclei	402	30	28	19	30	4	8	21	9	132	16
bl_chromatin	152	166	165	40	34	10	73	28	11	20	0
cell_shape	353	59	56	44	34	30	30	28	7	58	0
cell_size	384	45	52	40	30	27	19	29	6	67	0
cl_thickness	145	50	108	80	130	34	23	46	14	69	0
epith_c_size	47	386	72	48	39	41	12	21	2	31	0
marg_adhesion	407	58	58	33	23	22	13	25	5	55	0
mitoses	579	35	33	12	6	3	9	8	0	14	0
normal_nucleoli	443	36	44	18	19	22	16	24	16	61	0

```

df |> count(class) |> mutate(prop = round(n / sum(n), 3))

```

```

# A tibble: 2 × 3
  class      n prop
<chr>    <int> <dbl>
1 benign    458 0.655
2 malignant  241 0.345

```

Problems detected

- All predictors are stored as numbers (`dbl`), but they are really **ordinal scores**, so they should be integers.

- The frequency table shows an NA column: `bare_nuclei` has missing values (see next step).
- The predictors are scored on a **1–10 scale**, with no values out of range.
- `class` is text; it should be a factor with `benign` as the reference level. The classes are imbalanced (about 66% benign / 34% malignant), which is worth remembering for later.

2.4 Missing values

```
# Missing values per column
df |>
  summarise(across(everything(), ~ sum(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "n_missing") |>
  filter(n_missing > 0)
```

```
# A tibble: 1 × 2
  variable    n_missing
  <chr>         <int>
1 bare_nuclei      16
```

Problem detected: `bare_nuclei` has 16 missing values, and no other column has any. That is about 2.3% of the rows. Before deciding what to do with them, let's see whether they relate to the tumour class.

```
df |>
  filter(is.na(bare_nuclei)) |>
  count(class)
```

```
# A tibble: 2 × 2
  class      n
  <chr>    <int>
1 benign    14
2 malignant  2
```

14 of the 16 missing cases are benign (88%, versus 66% overall). That is a bit higher than expected, but with only 16 cases it is weak evidence, and removing them barely changes the class balance. Because K-Means and Random Forests (in their standard implementations) need complete rows and only 2.3% of the data is affected, the 16 rows are removed (**listwise deletion**). This decision is documented here so it can be revisited (e.g., with imputation) if needed.

```
df <- df |>
  drop_na(bare_nuclei)

nrow(df)
```

```
[1] 683
```

```
anyNA(df)
```

```
[1] FALSE
```

2.5 Duplicated records

```
# Repeated patient ids
df |>
  add_count(id, name = "n_id") |>
  filter(n_id > 1) |>
  arrange(id) |>
  head(12)
```

```
# A tibble: 12 × 12
   id      cl_thickness cell_size cell_shape marg_adhesion epith_c_size
  <chr>          <dbl>     <dbl>     <dbl>         <dbl>         <dbl>
1 1017023          4         1         1             3             2
2 1017023          6         3         3             5             3
3 1033078          2         1         1             1             2
4 1033078          4         2         1             1             2
5 1061990          1         1         3             2             2
6 1061990          4         1         1             1             2
7 1070935          1         1         3             1             2
8 1070935          3         1         1             1             1
9 1100524          6        10        10             2             8
10 1100524          6        10        10             2             8
11 1105524          1         1         1             1             2
12 1105524          4         1         1             1             2
# i 6 more variables: bare_nuclei <dbl>, bl_chromatin <dbl>,
#   normal_nucleoli <dbl>, mitoses <dbl>, class <chr>, n_id <int>
```

```
df |> summarise(rows = n(), unique_ids = n_distinct(id))
```

```
# A tibble: 1 × 2
  rows unique_ids
  <int>     <int>
1   683       630
```

```
# How many of the repeated ids are fully identical rows?
df |> get_dupes() |> distinct() |> nrow() # distinct duplicated profiles
```

No variable names specified - using all columns.

```
[1] 8
```

```
df |> get_dupes() |> count(dupe_count)
```

No variable names specified - using all columns.

```
# A tibble: 1 × 2
  dupe_count      n
    <int> <int>
1         2    16
```

Problem detected: some ids appear more than once. Two different situations are mixed:

1. **Exact duplicates** (every variable is identical): these are most likely data-entry duplicates and would be counted twice in the analysis.
2. **Same id, different measurements:** the same patient can have several samples taken at different times, so these are legitimate observations and are kept.

```
n_before <- nrow(df)
df <- df |> distinct()
tibble(before = n_before, after = nrow(df), removed = n_before - nrow(df))
```

```
# A tibble: 1 × 3
  before after removed
    <int> <int>   <int>
1    683   675       8
```

Only exact duplicates are removed. The repeated ids that remain have different measurements.

2.6 Coding the variables

`class` becomes a factor (a requirement for Random Forest classification) and the 1-10 scores become integers. The scores stay numeric, which is what K-Means needs.

```
df <- df |>
  mutate(
```



```

    class = factor(class, levels = c("benign", "malignant")),
    across(-c(id, class), as.integer)
  )

str(df)

```

```

tibble [675 × 11] (S3: tbl_df/tbl/data.frame)
 $ id          : chr [1:675] "1000025" "1002945" "1015425" "1016277" ...
 $ cl_thickness : int [1:675] 5 5 3 6 4 8 1 2 2 4 ...
 $ cell_size    : int [1:675] 1 4 1 8 1 10 1 1 1 2 ...
 $ cell_shape   : int [1:675] 1 4 1 8 1 10 1 2 1 1 ...
 $ marg_adhesion : int [1:675] 1 5 1 1 3 8 1 1 1 1 ...
 $ epith_c_size  : int [1:675] 2 7 2 3 2 7 2 2 2 2 ...
 $ bare_nuclei   : int [1:675] 1 10 2 4 1 10 10 1 1 1 ...
 $ bl_chromatin  : int [1:675] 3 3 3 3 3 9 3 3 1 2 ...
 $ normal_nucleoli : int [1:675] 1 2 1 7 1 7 1 1 1 1 ...
 $ mitoses       : int [1:675] 1 1 1 1 1 1 1 1 5 1 ...
 $ class         : Factor w/ 2 levels "benign","malignant": 1 1 1 1 1 2 1 1
 1 1 ...

```

2.7 Final check

```

# Everything the previous steps found should now be clean
checks <- tibble(
  check = c("No missing values",
            "No exact duplicate rows",
            "All scores in 1-10",
            "Class has 2 levels"),
  passed = c(!anyNA(df),
             !anyDuplicated(df),
             all(df |> select(-id, -class) |> unlist() |> between(1, 10)),
             nlevels(df$class) == 2)
)

kable(checks)

```

check	passed
No missing values	TRUE
No exact duplicate rows	TRUE
All scores in 1-10	TRUE
Class has 2 levels	TRUE

```
dim(df)
```

```
[1] 675 11
```

```
df |> count(class)
```

```
# A tibble: 2 × 2
  class      n
  <fct>    <int>
1 benign    439
2 malignant 236
```

```
dir.create("../data/clean", showWarnings = FALSE)
write_csv(df, "../data/clean/breast_cancer_clean.csv")
```

3. Initial Statistical Description

From here on, the cleaned data (`df`) is used: 675 observations, 9 numeric predictors scored 1-10, and the tumour `class`.

3.1 Class distribution

```
df |>
  count(class) |>
  mutate(percent = round(100 * n / sum(n), 1)) |>
  kable()
```

class	n	percent
benign	439	65
malignant	236	35

About two thirds of the cases are benign, so the classes are moderately imbalanced. K-Means does not use the class, but the imbalance matters when evaluating the clusters and the Random Forest: accuracy alone can look good just by favouring the benign class, so sensitivity and specificity are reported as well.

3.2 Summary statistics of the predictors

```
predictors <- df |> select(-id, -class)

predictors |>
```

```

pivot_longer(everything(), names_to = "variable") |>
group_by(variable) |>
summarise(
  mean = mean(value),
  sd = sd(value),
  median = median(value)
) |>
arrange(desc(mean)) |>
kable(digits = 2)

```

variable	mean	sd	median
cl_thickness	4.45	2.82	4
bare_nuclei	3.54	3.64	1
bl_chromatin	3.44	2.45	3
epith_c_size	3.23	2.21	2
cell_shape	3.21	2.98	1
cell_size	3.15	3.06	1
normal_nucleoli	2.89	3.07	1
marg_adhesion	2.85	2.88	1
mitoses	1.61	1.74	1

Observations

- All variables use the same 1-10 scale, and most have a median at or near the minimum (1 for `cell_size`, `cell_shape`, `marg_adhesion`, `bare_nuclei`, `normal_nucleoli` and `mitoses`) with a mean well above it. Most tumours score low, and a smaller group scores high.

3.3 Predictors by tumour class

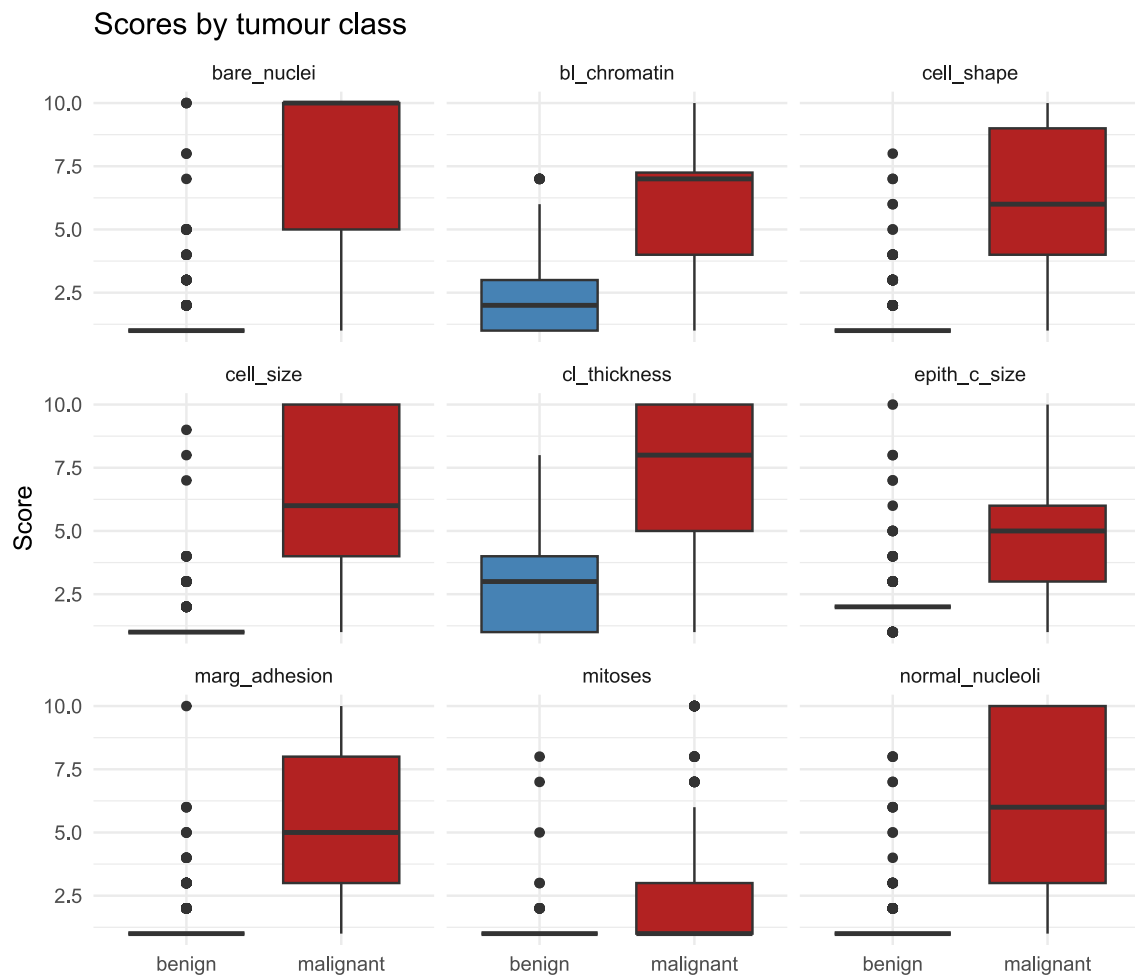
```

df |>
group_by(class) |>
summarise(across(-id, mean)) |>
pivot_longer(-class, names_to = "variable") |>
pivot_wider(names_from = class, values_from = value) |>
mutate(difference = malignant - benign) |>
arrange(desc(difference)) |>
kable(digits = 2)

```

variable	benign	malignant	difference
bare_nuclei	1.35	7.61	6.26
cell_size	1.31	6.56	5.25
cell_shape	1.42	6.54	5.12
normal_nucleoli	1.26	5.90	4.64
marg_adhesion	1.35	5.64	4.28
cl_thickness	2.97	7.20	4.23
bl_chromatin	2.08	5.98	3.90
epith_c_size	2.11	5.31	3.19
mitoses	1.07	2.61	1.55

```
df |>
  pivot_longer(-c(id, class), names_to = "variable", values_to = "score") |>
  ggplot(aes(x = class, y = score, fill = class)) +
  geom_boxplot(show.legend = FALSE) +
  facet_wrap(~ variable) +
  scale_fill_manual(values = c(benign = "steelblue", malignant = "firebrick")) +
  labs(x = NULL, y = "Score", title = "Scores by tumour class") +
  theme_minimal()
```



Observations

- Malignant tumours score higher on **every** variable. The largest gaps are in `bare_nuclei` (about 1.4 in benign vs 7.6 in malignant), `cl_thickness`, `cell_size` and `cell_shape`.
- `mitoses` has the smallest gap, and most of its values are 1 in both classes.
- Since every variable separates the classes to some degree, both K-Means and the Random Forest have information to work with.

3.4 Correlations between predictors

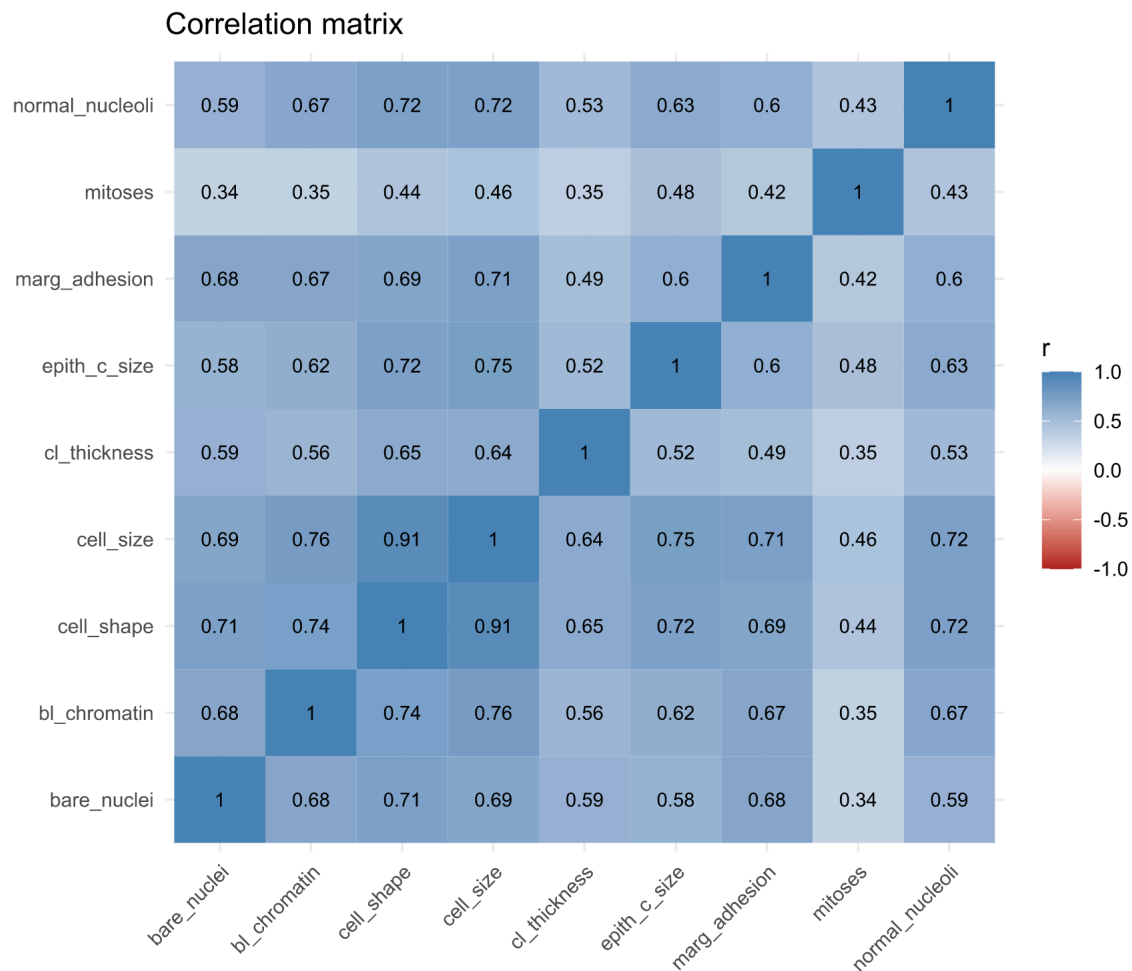
```
cor_mat <- cor(predictors)

cor_mat |>
  as_tibble(rownames = "var1") |>
  pivot_longer(-var1, names_to = "var2", values_to = "r") |>
  ggplot(aes(var1, var2, fill = r)) +
  geom_tile() +
```

```

geom_text(aes(label = round(r, 2)), size = 3) +
scale_fill_gradient2(low = "firebrick", mid = "white", high = "steelblue",
                     midpoint = 0, limits = c(-1, 1)) +
labs(x = NULL, y = NULL, fill = "r", title = "Correlation matrix") +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



```

# Strongest and weakest pairs
cor_mat |>
  as_tibble(rownames = "var1") |>
  pivot_longer(-var1, names_to = "var2", values_to = "r") |>
  filter(var1 < var2) |>
  arrange(desc(r)) |>
  slice(c(1:3, (n() - 2):n())) |>
  kable(digits = 2)

```

var1	var2	r
cell_shape	cell_size	0.91
bl_chromatin	cell_size	0.76
cell_size	epith_c_size	0.75
cl_thickness	mitoses	0.35
bl_chromatin	mitoses	0.35
bare_nuclei	mitoses	0.34

Observations

- All correlations are **positive**, and most are moderate to strong (0.5-0.9).
- The highest correlation is between `cell_size` and `cell_shape` (0.91); these two are nearly redundant.
- `mitoses` is the least correlated with the rest (0.34-0.48).

4. K-Means Algorithm

K-Means is applied to the nine predictors (`id` and `class` are excluded) with $K = 2$, and the clusters are then compared with the real tumour classes. The class is never used to fit the model, so this checks whether the two groups are naturally present in the data.

4.1 Fitting the model

All variables share the same 1-10 scale, so they are used **without standardising**; scaling would only inflate the weight of low-variance variables such as `mitoses`. K-Means depends on the random initial centres, so the seed is fixed (`set.seed(123)`) and `nstart = 25` starting configurations are tried, keeping the best one.

```
set.seed(123)
km <- kmeans(predictors, centers = 2, nstart = 25)

km$size
```

```
[1] 448 227
```

```
round(km$centers, 2)
```

```
  cl_thickness cell_size cell_shape marg_adhesion epith_c_size bare_nuclei
1         3.06      1.30      1.43          1.36          2.10          1.32
2         7.19      6.79      6.71          5.79          5.46          7.91
  bl_chromatin normal_nucleoli mitoses
```

1	2.09	1.26	1.11
2	6.11	6.09	2.58

```
round(100 * km$betweenss / km$totss, 1) # % of variance explained by the clusters
```

```
[1] 60.2
```

The cluster numbers (1 and 2) are arbitrary. One cluster has low scores on every variable and the other has high scores, which looks like a benign / malignant split. To compare with the real classes, the cluster with the higher average centre is labelled `malignant`.

```
malignant_cluster <- which.max(rowMeans(km$centers))

df_km <- df |>
  mutate(cluster = factor(km$cluster,
                          levels = c(setdiff(1:2, malignant_cluster),
                                      malignant_cluster),
                          labels = c("benign", "malignant")))

count(df_km, cluster)
```

```
# A tibble: 2 × 2
  cluster      n
  <fct>    <int>
1 benign    448
2 malignant  227
```

4.2 Comparison with the original classes

```
conf <- table(Class = df_km$cluster, Class = df_km$class)
conf
```

Cluster	Class	
	benign	malignant
benign	430	18
malignant	9	218

```
round(100 * prop.table(conf, margin = 1), 1) # % of each cluster by real class
```


	Class	
Cluster	benign	malignant
benign	96	4
malignant	4	96

```
# Treating "malignant" as the positive class
tp <- conf["malignant", "malignant"]
tn <- conf["benign", "benign"]
fp <- conf["malignant", "benign"]
fn <- conf["benign", "malignant"]

tibble(
  metric = c("Accuracy", "Sensitivity (malignant found)",
            "Specificity (benign found)", "Precision (malignant)"),
  value = c((tp + tn) / sum(conf), tp / (tp + fn), tn / (tn + fp), tp / (tp
+ fp))
) |>
  mutate(value = scales::percent(value, accuracy = 0.1)) |>
  kable()
```

metric	value
Accuracy	96.0%
Sensitivity (malignant found)	92.4%
Specificity (benign found)	97.9%
Precision (malignant)	96.0%

```
library(cluster)
sil <- silhouette(km$cluster, dist(predictors))
mean(sil[, "sil_width"])
```

```
[1] 0.5968713
```

4.3 Visualisation

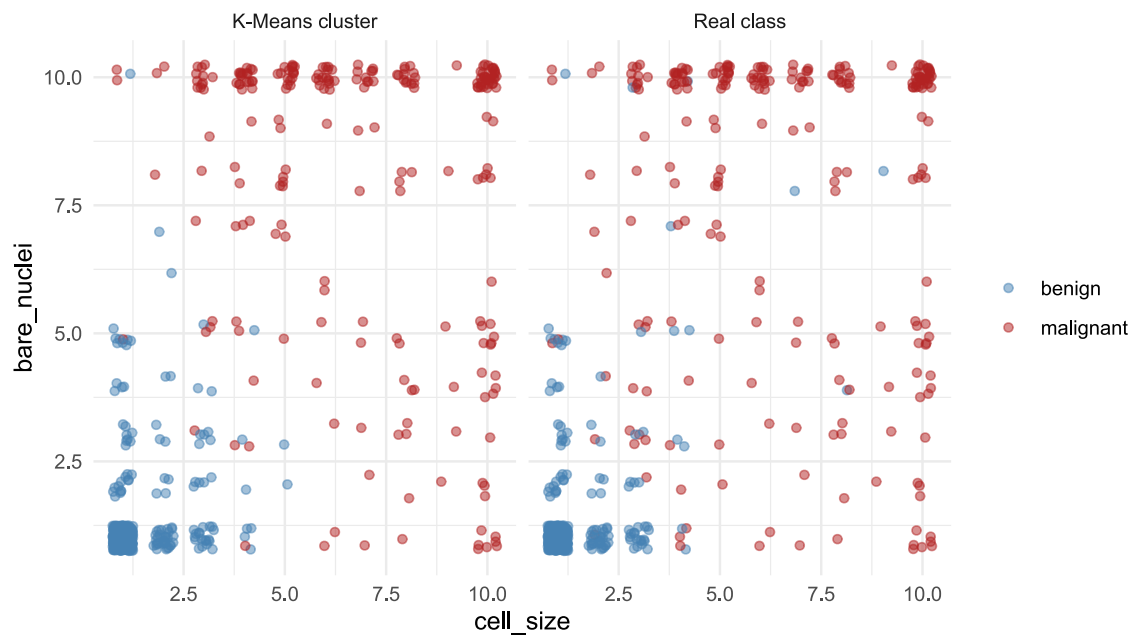
The clusters are drawn on two of the most discriminating variables (`cell_size` and `bare_nuclei`), next to the real classes. Points are jittered because the scores are integers.

```
bind_rows(
  mutate(df_km, grouping = "K-Means cluster", group = cluster),
  mutate(df_km, grouping = "Real class", group = class)
```

```

) |>
  ggplot(aes(cell_size, bare_nuclei, colour = group)) +
  geom_jitter(alpha = 0.5, width = 0.25, height = 0.25) +
  facet_wrap(~ grouping) +
  scale_colour_manual(values = c(benign = "steelblue", malignant = "firebrick"))
+
  labs(colour = NULL) +
  theme_minimal()

```



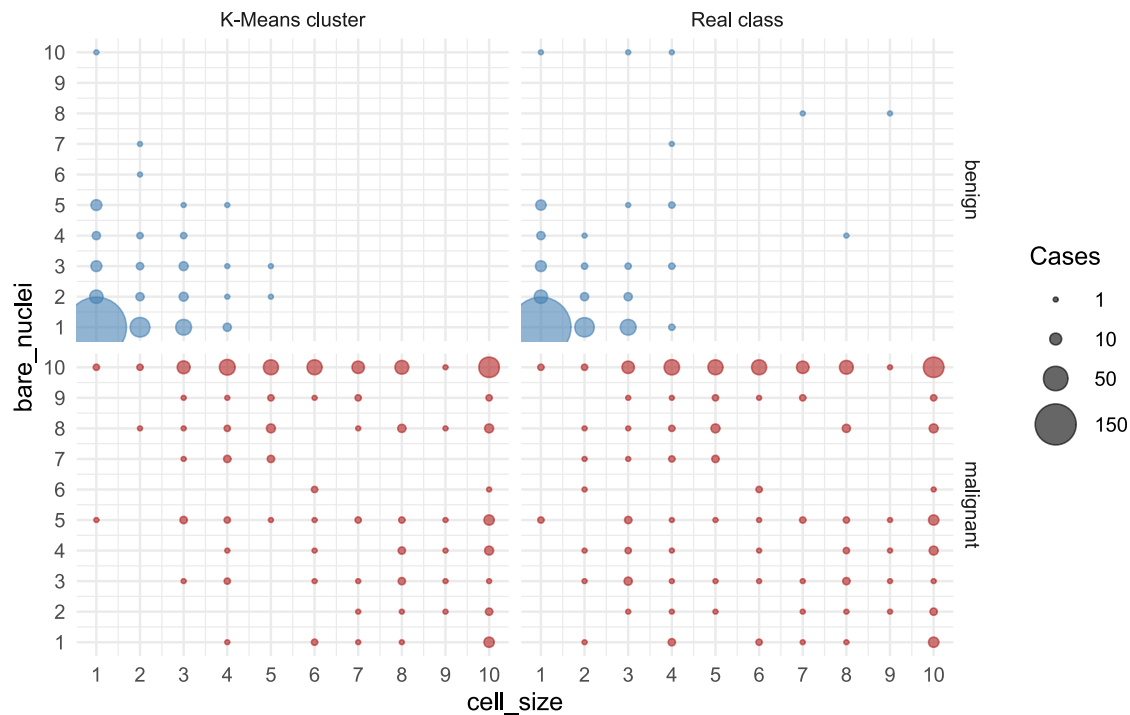
Many cases share exactly the same pair of scores (for example, about 400 tumours score 1 on both variables), so the jittered points overlap. Drawing the **number of cases** at each combination of scores makes the volume visible:

```

bind_rows(
  mutate(df_km, grouping = "K-Means cluster", group = cluster),
  mutate(df_km, grouping = "Real class", group = class)
) |>
  count(grouping, group, cell_size, bare_nuclei) |>
  ggplot(aes(cell_size, bare_nuclei, size = n, colour = group)) +
  geom_point(alpha = 0.6) +
  facet_grid(group ~ grouping) +
  scale_size_area(max_size = 12, breaks = c(1, 10, 50, 150)) +
  scale_x_continuous(breaks = 1:10) +
  scale_y_continuous(breaks = 1:10) +

```

```
scale_colour_manual(values = c(benign = "steelblue", malignant = "firebrick"),
                    guide = "none") +
labs(size = "Cases") +
theme_minimal()
```



Each row shows one group (benign or malignant) and each column shows how that group was defined (by K-Means or by the real class), so the two columns can be compared cell by cell.

Observations

- K-Means finds two groups of 448 and 227 cases, which are close to the real class sizes (439 benign and 236 malignant). The clusters explain about 60% of the total variance.
- Overall, 96% of the cases are placed in the group that matches their real class (accuracy 96.0%): 430 benign and 218 malignant tumours are correctly grouped.
- The 27 mismatches are not symmetric: 18 malignant tumours fall in the benign cluster and 9 benign ones in the malignant cluster. The sensitivity (92.4%) is therefore lower than the specificity (97.9%).
- The average silhouette (about 0.6) indicates a reasonably well-separated structure: the groups are distinct, with some overlap in the middle.

5. Random Forest

A Random Forest is trained to predict `class` from the nine predictors, using 70% of the data for training and 30% for testing.

5.1 Train / test split

Some patients have more than one sample (section 2.5). If the same patient appeared in both sets, the test would partly measure memory instead of generalisation. The split is therefore done **by patient id**: all samples of a patient go to the same set.

```
library(rsample)

set.seed(123)
split <- group_initial_split(df, group = id, prop = 0.7)
train <- training(split)
test  <- testing(split)

tibble(set = c("train", "test"), rows = c(nrow(train), nrow(test))) |>
  mutate(percent = round(100 * rows / sum(rows), 1))
```

```
# A tibble: 2 × 3
  set    rows percent
<chr> <int>   <dbl>
1 train   472    69.9
2 test   203    30.1
```

```
length(intersect(train$id, test$id)) # patients shared by both sets
```

```
[1] 0
```

```
bind_rows(train = train, test = test, .id = "set") |>
  count(set, class) |>
  group_by(set) |>
  mutate(percent = round(100 * n / sum(n), 1)) |>
  kable()
```

set	class	n	percent
test	benign	138	68.0
test	malignant	65	32.0
train	benign	301	63.8
train	malignant	171	36.2

The split is 70 / 30, no patient is in both sets, and both sets keep a class balance close to the full data (about 65% benign).

5.2 Fitting the model

The default settings are used: 500 trees and 3 variables tried at each split (the square root of 9). `id` is excluded from the predictors.

```
library(randomForest)
```

```
randomForest 4.7-1.2
```

```
Type rfNews() to see new features/changes/bug fixes.
```

```
Attaching package: 'randomForest'
```

```
The following object is masked from 'package:dplyr':
```

```
combine
```

```
The following object is masked from 'package:ggplot2':
```

```
margin
```

```
set.seed(123)
rf <- randomForest(class ~ . - id, data = train, ntree = 500, importance = TRUE)
rf
```

```
Call:
randomForest(formula = class ~ . - id, data = train, ntree = 500, importance
= TRUE)
```

```
      Type of random forest: classification
```

```
      Number of trees: 500
```

```
No. of variables tried at each split: 3
```

```
      OOB estimate of  error rate: 3.18%
```

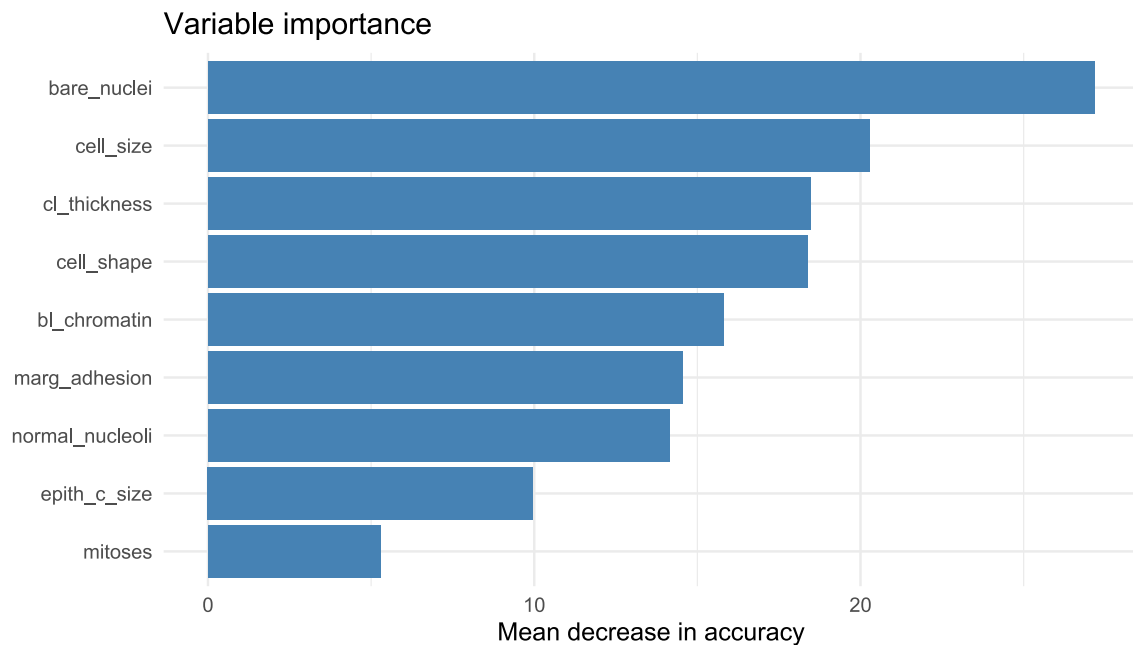
```
Confusion matrix:
```

	benign	malignant	class.error
benign	292	9	0.02990033
malignant	6	165	0.03508772

The out-of-bag (OOB) error, estimated only from the training data, is about 3%.

5.3 Variable importance

```
importance(rf) |>
  as_tibble(rownames = "variable") |>
  ggplot(aes(MeanDecreaseAccuracy, reorder(variable, MeanDecreaseAccuracy))) +
  geom_col(fill = "steelblue") +
  labs(x = "Mean decrease in accuracy", y = NULL, title = "Variable importance") +
  theme_minimal()
```



`bare_nuclei` is the most important variable, followed by `cell_size`, `cl_thickness` and `cell_shape`. `mitoses` is the least important.

6. Performance Metrics

The model predicts the test set, which it has never seen.

```
pred <- predict(rf, test)

rf_conf <- table(Predicted = pred, Class = test$class)
rf_conf
```

	Class	
Predicted	benign	malignant
benign	136	5
malignant	2	60

```
mean(pred == test$class) # accuracy
```

```
[1] 0.9655172
```

Each metric depends on which class is taken as positive, so they are computed for **both classes**:

```
class_metrics <- function(conf, positive) {  
  tp <- conf[positive, positive]  
  fn <- sum(conf[, positive]) - tp  
  fp <- sum(conf[positive, ]) - tp  
  tn <- sum(conf) - tp - fn - fp  
  sensitivity <- tp / (tp + fn)  
  precision <- tp / (tp + fp)  
  tibble(  
    class = positive,  
    sensitivity = sensitivity,  
    specificity = tn / (tn + fp),  
    precision = precision,  
    f1 = 2 * precision * sensitivity / (precision + sensitivity)  
  )  
}  
  
map(levels(test$class), \(k) class_metrics(rf_conf, k)) |>  
  list_rbind() |>  
  mutate(across(-class, \(x) scales::percent(x, accuracy = 0.1))) |>  
  kable()
```

class	sensitivity	specificity	precision	f1
benign	98.6%	92.3%	96.5%	97.5%
malignant	92.3%	98.6%	96.8%	94.5%

Observations

- The accuracy on the test set is 96.6%: 196 of 203 cases are correct (136 benign and 60 malignant).
- Only 7 cases are wrong: 5 malignant tumours are predicted as benign and 2 benign ones as malignant.
- The sensitivity of one class equals the specificity of the other, because both come from the same two kinds of error.

7. Discussion

7.1 Best and worst performance

- **Best:** the model finds almost every benign tumour (sensitivity 98.6%) and, equally, rarely labels a benign tumour as malignant (malignant specificity 98.6%). The benign class also has the highest F1 (97.5%).
- **Worst:** the sensitivity for malignant tumours (92.3%) is the lowest metric. 5 of the 65 malignant tumours in the test set are missed, which also gives the malignant class the lower F1 (94.5%).
- Precision is high and almost equal for both classes (96.5% and 96.8%): when the model says “malignant”, it is right about 97% of the time.

7.2 Interpretation

- **The weak spot is the one that matters most.** In a diagnostic setting, a missed malignant tumour (false negative) is far more costly than a false alarm, which is usually caught by a follow-up test. The model makes this costly error more often than the opposite one. Part of the reason is the class imbalance: there are fewer malignant cases to learn from.
- **The important variables match the initial description.** `bare_nuclei`, `cell_size`, `cell_shape` and `cl_thickness` had the largest gaps between classes in section 3.3, and they are also the most useful for the forest. `mitoses`, which was mostly 1 in both classes, adds little.
- **Random Forest vs K-Means.** The forest (accuracy 96.6%) is only slightly better than K-Means (96.0%), and both miss malignant tumours more often than benign ones (malignant sensitivity about 92% in both). With 65 malignant cases in the test set, each case is worth about 1.5 points, so the difference between the two models is small. The data has a clear two-group structure that even an unsupervised method finds, so the main gain of the supervised model is that it can be evaluated on new cases and tuned, not a large jump in accuracy.
- **Possible improvement.** If missing a malignant tumour is the main concern, the decision threshold can be lowered (classify as malignant with less than 50% of the tree votes). This raises malignant sensitivity at the cost of more false alarms.